

Модуль шифрования Гронсвельда

Создано системой Doxygen 1.9.4

1	Алфавитный указатель классов	1
1.1	Классы	1
2	Список файлов	3
2.1	Файлы	3
3	Классы	5
3.1	Класс <code>modAlphaCipher</code>	5
3.1.1	Подробное описание	5
3.1.2	Конструктор(ы)	5
3.1.2.1	<code>modAlphaCipher()</code> [1/2]	6
3.1.2.2	<code>modAlphaCipher()</code> [2/2]	6
3.1.3	Методы	6
3.1.3.1	<code>decrypt()</code>	6
3.1.3.2	<code>encrypt()</code>	7
4	Файлы	9
4.1	Файл <code>main.cpp</code>	9
4.1.1	Подробное описание	10
4.1.2	Функции	10
4.1.2.1	<code>main()</code>	10
4.1.2.2	<code>remove_spaces()</code>	10
4.1.2.3	<code>string_to_wstring()</code>	11
4.1.2.4	<code>wstring_to_string()</code>	11
4.2	<code>main.cpp</code>	11
4.3	Файл <code>modAlphaCipher.cpp</code>	13
4.3.1	Подробное описание	13
4.4	<code>modAlphaCipher.cpp</code>	13
4.5	Файл <code>modAlphaCipher.h</code>	14
4.5.1	Подробное описание	15
4.6	<code>modAlphaCipher.h</code>	16
	Предметный указатель	17

Глава 1

Алфавитный указатель классов

1.1 Классы

Классы с их кратким описанием.

[modAlphaCipher](#)

Класс для шифрования и дешифрования текста методом Гронсвельда [5](#)

Глава 2

Список файлов

2.1 Файлы

Полный список файлов.

main.cpp	Главный файл программы шифрования Гронсвельда	9
modAlphaCipher.cpp	Реализация класса modAlphaCipher	13
modAlphaCipher.h	Заголовочный файл для класса modAlphaCipher	14

Глава 3

Классы

3.1 Класс `modAlphaCipher`

Класс для шифрования и дешифрования текста методом Гронсвельда

```
#include <modAlphaCipher.h>
```

Открытые члены

- `modAlphaCipher ()=delete`
Удаленный конструктор по умолчанию Класс не может быть создан без ключа
- `modAlphaCipher (const std::wstring &skey)`
Конструктор класса `modAlphaCipher`.
- `std::wstring encrypt (const std::wstring &open_text)`
Шифрует открытый текст
- `std::wstring decrypt (const std::wstring &cipher_text)`
Дешифрует зашифрованный текст

3.1.1 Подробное описание

Класс для шифрования и дешифрования текста методом Гронсвельда

Класс реализует алгоритм шифрования Гронсвельда с поддержкой русского алфавита. Шифрование осуществляется путем сдвига букв на величину, определяемую ключом.

См. определение в файле `modAlphaCipher.h` строка 24

3.1.2 Конструктор(ы)

3.1.2.1 modAlphaCipher() [1/2]

modAlphaCipher::modAlphaCipher () [delete]

Удаленный конструктор по умолчанию Класс не может быть создан без ключа

3.1.2.2 modAlphaCipher() [2/2]

modAlphaCipher::modAlphaCipher (
 const std::wstring & skey)

Конструктор класса [modAlphaCipher](#).

Аргументы

skey	Ключ шифрования в виде строки
------	-------------------------------

Исключения

std::invalid_argument	если ключ содержит недопустимые символы
-----------------------	---

Аргументы

skey	Ключ шифрования в виде строки
------	-------------------------------

Исключения

std::invalid_argument	если ключ содержит недопустимые символы
-----------------------	---

Конструктор инициализирует алфавит, создает карту символов, проверяет и преобразует ключ, затем сохраняет его в числовом виде.

См. определение в файле [modAlphaCipher.cpp](#) строка [20](#)

3.1.3 Методы

3.1.3.1 decrypt()

std::wstring modAlphaCipher::decrypt (
 const std::wstring & cipher_text)

Дешифрует зашифрованный текст

Аргументы

cipher_text	Текст для дешифрования
-------------	------------------------

Возвращает

Расшифрованная строка

Исключения

std::invalid_argument	если текст содержит недопустимые символы
-----------------------	--

Аргументы

cipher_text	Текст для дешифрования
-------------	------------------------

Возвращает

Расшифрованная строка

Исключения

std::invalid_argument	если текст содержит недопустимые символы
-----------------------	--

Алгоритм дешифрования является обратным к алгоритму шифрования:

1. Преобразование текста в верхний регистр
2. Проверка валидности символов
3. Преобразование текста в числовой вид
4. Обратный сдвиг на величину ключа
5. Преобразование результата обратно в строку

См. определение в файле [modAlphaCipher.cpp](#) строка [75](#)

Используется в [main\(\)](#).

3.1.3.2 encrypt()

```
std::wstring modAlphaCipher::encrypt (
    const std::wstring & open_text )
```

Шифрует открытый текст

Аргументы

<code>open_text</code>	Текст для шифрования
------------------------	----------------------

Возвращает

Зашифрованная строка

Исключения

<code>std::invalid_argument</code>	если текст содержит недопустимые символы
------------------------------------	--

Аргументы

<code>open_text</code>	Текст для шифрования
------------------------	----------------------

Возвращает

Зашифрованная строка

Исключения

<code>std::invalid_argument</code>	если текст содержит недопустимые символы
------------------------------------	--

Алгоритм шифрования:

1. Преобразование текста в верхний регистр
2. Проверка валидности символов
3. Преобразование текста в числовой вид
4. Применение шифра Гронсвельда (сдвиг на величину ключа)
5. Преобразование результата обратно в строку

См. определение в файле [modAlphaCipher.cpp](#) строка 48

Используется в [main\(\)](#).

Объявления и описания членов классов находятся в файлах:

- [modAlphaCipher.h](#)
- [modAlphaCipher.cpp](#)

Глава 4

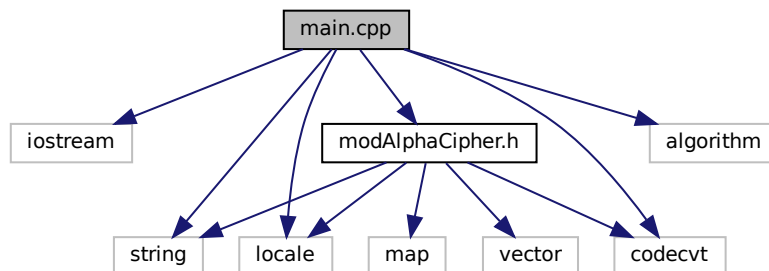
Файлы

4.1 Файл main.cpp

Главный файл программы шифрования Гронсвельда

```
#include <iostream>
#include <string>
#include <locale>
#include <codecvt>
#include <algorithm>
#include "modAlphaCipher.h"
```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- std::wstring [string_to_wstring](#) (const std::string &str)
Преобразует string в wstring.
- std::string [wstring_to_string](#) (const std::wstring &wstr)
Преобразует wstring в string.
- std::wstring [remove_spaces](#) (const std::wstring &str)
Удаляет пробелы из строки
- int [main](#) ()
Главная функция программы

4.1.1 Подробное описание

Главный файл программы шифрования Гронсвельда

Содержит точку входа в программу и пользовательский интерфейс

См. определение в файле [main.cpp](#)

4.1.2 Функции

4.1.2.1 main()

```
int main ( )
```

Главная функция программы

Возвращает

Код завершения программы (0 - успешно, 1 - ошибка)

Программа предоставляет интерактивный интерфейс для:

1. Ввода ключа шифрования
2. Выбора операции (шифрование/дешифрование)
3. Ввода текста для обработки
4. Вывода результата

См. определение в файле [main.cpp](#) строка 55

Перекрестные ссылки [modAlphaCipher::decrypt\(\)](#), [modAlphaCipher::encrypt\(\)](#) и [remove_spaces\(\)](#).

4.1.2.2 remove_spaces()

```
std::wstring remove_spaces (  
    const std::wstring & str )
```

Удаляет пробелы из строки

Аргументы

str	Входная строка
-----	----------------

Возвращает

Строка без пробелов

См. определение в файле [main.cpp](#) строка 39

Используется в [main\(\)](#).

4.1.2.3 string_to_wstring()

```
std::wstring string_to_wstring (
    const std::string & str )
```

Преобразует string в wstring.

Аргументы

str	Строка для преобразования
-----	---------------------------

Возвращает

Широкая строка (wstring)

См. определение в файле [main.cpp](#) строка 19

4.1.2.4 wstring_to_string()

```
std::string wstring_to_string (
    const std::wstring & wstr )
```

Преобразует wstring в string.

Аргументы

wstr	Широкая строка для преобразования
------	-----------------------------------

Возвращает

Обычная строка (string)

См. определение в файле [main.cpp](#) строка 29

4.2 main.cpp

[См. документацию.](#)

```

00001
00007 #include <iostream>
00008 #include <string>
00009 #include <locale>
00010 #include <codecvt>
00011 #include <algorithm>
00012 #include "modAlphaCipher.h"
00013
00019 std::wstring string_to_wstring(const std::string& str) {
00020     std::wstring_convert<std::codecvt_utf8<wchar_t>> converter;
00021     return converter.from_bytes(str);
00022 }
00023
00029 std::string wstring_to_string(const std::wstring& wstr) {
00030     std::wstring_convert<std::codecvt_utf8<wchar_t>> converter;
00031     return converter.to_bytes(wstr);
00032 }
00033
00039 std::wstring remove_spaces(const std::wstring& str) {
00040     std::wstring result = str;
00041     result.erase(std::remove(result.begin(), result.end(), L' '), result.end());
00042     return result;
00043 }
00044
00055 int main()
00056 {
00057     // Устанавливаем локаль
00058     try {
00059         std::locale::global(std::locale("ru_RU.UTF-8"));
00060     } catch (const std::exception& e) {
00061         std::cout << "Не удалось установить локаль: " << e.what() << std::endl;
00062         std::cout << "Продолжаем без локали..." << std::endl;
00063     }
00064
00065     std::wcout.imbue(std::locale());
00066     std::wcin.imbue(std::locale());
00067
00068     std::wstring key;
00069     std::wstring text;
00070     int op;
00071
00072     std::wcout << L"Шифратор готов. Введите ключ (только русские буквы): ";
00073     std::getline(std::wcin, key);
00074
00075     // Удаляем пробелы из ключа
00076     key = remove_spaces(key);
00077
00078     try {
00079         modAlphaCipher cipher(key);
00080         std::wcout << L"Ключ загружен успешно." << std::endl;
00081
00082         do {
00083             std::wcout << L"\nВыберите операцию (0-выход, 1-зашифровать, 2-расшифровать): ";
00084             std::wcin >> op;
00085             std::wcin.ignore(); // Очищаем буфер
00086
00087             if (op > 2) {
00088                 std::wcout << L"Неверная операция!" << std::endl;
00089             } else if (op > 0) {
00090                 std::wcout << L"Введите текст (только русские буквы): ";
00091                 std::getline(std::wcin, text);
00092
00093                 // Удаляем пробелы из текста
00094                 text = remove_spaces(text);
00095
00096                 try {
00097                     if (op == 1) {
00098                         std::wstring encrypted = cipher.encrypt(text);
00099                         std::wcout << L"Зашифрованный текст: " << encrypted << std::endl;
00100                     } else {
00101                         std::wstring decrypted = cipher.decrypt(text);
00102                         std::wcout << L"Расшифрованный текст: " << decrypted << std::endl;
00103                     }
00104                 } catch (const std::exception& e) {
00105                     std::wcout << L"Ошибка при обработке: " << e.what() << std::endl;
00106                 }
00107             }
00108         } while (op != 0);
00109
00110     } catch (const std::exception& e) {
00111         std::wcout << L"Ошибка инициализации: " << e.what() << std::endl;
00112         std::wcout << L"Пожалуйста, используйте только русские буквы." << std::endl;
00113         return 1;
00114     }
00115
00116     std::wcout << L"Выход из программы." << std::endl;
00117     return 0;

```

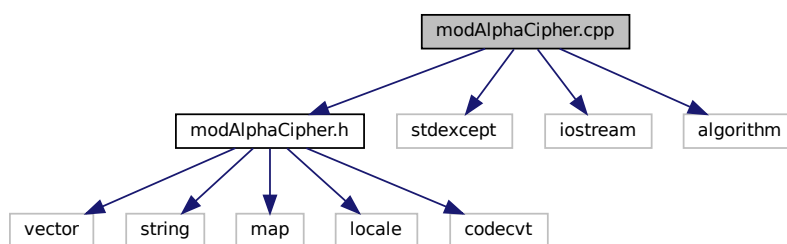
```
00118 }
```

4.3 Файл modAlphaCipher.cpp

Реализация класса `modAlphaCipher`.

```
#include "modAlphaCipher.h"
#include <stdexcept>
#include <iostream>
#include <algorithm>
```

Граф включаемых заголовочных файлов для modAlphaCipher.cpp:



4.3.1 Подробное описание

Реализация класса `modAlphaCipher`.

Содержит реализацию методов шифрования Гронсвельда

См. определение в файле `modAlphaCipher.cpp`

4.4 modAlphaCipher.cpp

См. документацию.

```

00001
00007 #include "modAlphaCipher.h"
00008 #include <stdexcept>
00009 #include <iostream>
00010 #include <algorithm>
00011
00020 modAlphaCipher::modAlphaCipher(const std::wstring& skey)
00021 {
00022     // Инициализируем алфавит и создаем карту символов
00023     for (size_t i = 0; i < numAlpha.size(); i++) {
00024         alphaNum[numAlpha[i]] = i;
00025     }
00026
00027     std::wstring upper_key = to_upper(skey);
00028     if (!isValid(upper_key)) {
00029         throw std::invalid_argument("Ключ содержит недопустимые символы");
00030     }
00031
00032     key = convert(upper_key);
00033 }
00034
00048 std::wstring modAlphaCipher::encrypt(const std::wstring& open_text)
```

```

00049 {
00050     std::wstring text = to_upper(open_text);
00051     if (!IsValid(text)) {
00052         throw std::invalid_argument("Текст содержит недопустимые символы");
00053     }
00054
00055     std::vector<int> work = convert(text);
00056     for (size_t i = 0; i < work.size(); i++) {
00057         work[i] = (work[i] + key[i % key.size()]) % numAlpha.size();
00058     }
00059     return convert(work);
00060 }
00061
00075 std::wstring modAlphaCipher::decrypt(const std::wstring& cipher_text)
00076 {
00077     std::wstring text = to_upper(cipher_text);
00078     if (!IsValid(text)) {
00079         throw std::invalid_argument("Текст содержит недопустимые символы");
00080     }
00081
00082     std::vector<int> work = convert(text);
00083     for (size_t i = 0; i < work.size(); i++) {
00084         work[i] = (work[i] + numAlpha.size() - key[i % key.size()]) % numAlpha.size();
00085     }
00086     return convert(work);
00087 }
00088
00095 std::vector<int> modAlphaCipher::convert(const std::wstring& s)
00096 {
00097     std::vector<int> result;
00098     for (wchar_t c : s) {
00099         if (alphaNum.find(c) == alphaNum.end()) {
00100             throw std::invalid_argument("Недопустимый символ в тексте");
00101         }
00102         result.push_back(alphaNum[c]);
00103     }
00104     return result;
00105 }
00106
00113 std::wstring modAlphaCipher::convert(const std::vector<int>& v)
00114 {
00115     std::wstring result;
00116     for (int i : v) {
00117         if (i < 0 || i >= static_cast<int>(numAlpha.size())) {
00118             throw std::out_of_range("Индекс выходит за границы алфавита");
00119         }
00120         result += numAlpha[i];
00121     }
00122     return result;
00123 }
00124
00130 std::wstring modAlphaCipher::to_upper(const std::wstring& s)
00131 {
00132     std::wstring result = s;
00133     std::wstring lower = L"абвгдеёжзийклмнопрстуфхцщъыьэюя";
00134     std::wstring upper = L"АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦШЩЪЫЬЭЮЯ";
00135
00136     for (size_t i = 0; i < result.size(); i++) {
00137         size_t pos = lower.find(result[i]);
00138         if (pos != std::wstring::npos) {
00139             result[i] = upper[pos];
00140         }
00141     }
00142
00143     // Удаляем пробелы из результата
00144     result.erase(std::remove(result.begin(), result.end(), L' '), result.end());
00145
00146     return result;
00147 }
00148
00154 bool modAlphaCipher::IsValid(const std::wstring& s)
00155 {
00156     for (wchar_t c : s) {
00157         if (alphaNum.find(c) == alphaNum.end()) {
00158             return false;
00159         }
00160     }
00161     return true;
00162 }

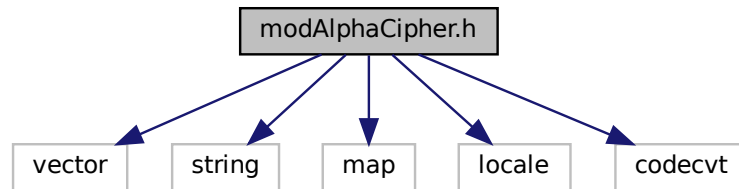
```

4.5 Файл modAlphaCipher.h

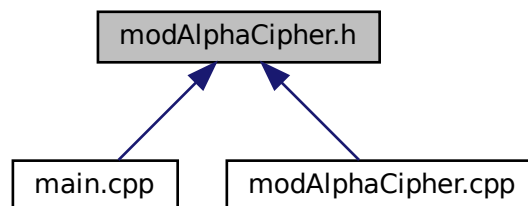
Заголовочный файл для класса `modAlphaCipher`.

```
#include <vector>
#include <string>
#include <map>
#include <locale>
#include <codecvt>
```

Граф включаемых заголовочных файлов для modAlphaCipher.h:



Граф файлов, в которые включается этот файл:



Классы

- class [modAlphaCipher](#)

Класс для шифрования и дешифрования текста методом Гронсвельда

4.5.1 Подробное описание

Заголовочный файл для класса [modAlphaCipher](#).

Реализация шифра Гронсвельда для русскоязычных сообщений

Версия

1.0

Дата

2025

Автор

Ходжаев С.А.

См. определение в файле [modAlphaCipher.h](#)

4.6 modAlphaCipher.h

[См. документацию.](#)

```
00001
00010 #pragma once
00011 #include <vector>
00012 #include <string>
00013 #include <map>
00014 #include <locale>
00015 #include <codecvt>
00016
00024 class modAlphaCipher
00025 {
00026 private:
00027     std::wstring numAlpha = L"АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ";
00028     std::map<wchar_t, int> alphaNum;
00029     std::vector<int> key;
00030
00037     std::vector<int> convert(const std::wstring& s);
00038
00045     std::wstring convert(const std::vector<int>& v);
00046
00052     std::wstring to_upper(const std::wstring& s);
00053
00059     bool isValid(const std::wstring& s);
00060
00061 public:
00066     modAlphaCipher() = delete;
00067
00073     modAlphaCipher(const std::wstring& skey);
00074
00081     std::wstring encrypt(const std::wstring& open_text);
00082
00089     std::wstring decrypt(const std::wstring& cipher_text);
00090 };
```

Предметный указатель

- decrypt
 - modAlphaCipher, [6](#)
- encrypt
 - modAlphaCipher, [7](#)
- main
 - main.cpp, [10](#)
- main.cpp, [9](#)
 - main, [10](#)
 - remove_spaces, [10](#)
 - string_to_wstring, [11](#)
 - wstring_to_string, [11](#)
- modAlphaCipher, [5](#)
 - decrypt, [6](#)
 - encrypt, [7](#)
 - modAlphaCipher, [5](#), [6](#)
- modAlphaCipher.cpp, [13](#)
- modAlphaCipher.h, [14](#)
- remove_spaces
 - main.cpp, [10](#)
- string_to_wstring
 - main.cpp, [11](#)
- wstring_to_string
 - main.cpp, [11](#)